

8. Distributed Operating System

1. Distributed Operating System Concept,
 2. advantages and types of distributed operating system,
 3. Design issues in distributed operating system, Communication and synchronization,
 4. Client-server computing,
 5. Algorithms for distributed control (Mutual exclusion, deadlock) Distributed file system,
 6. Security
-

8. Distributed Operating System

1. Distributed Operating System Concept

A Distributed Operating System (DOS) manages a group of independent computers and makes them appear to the user as a single unified system. It ensures resource sharing, load balancing, fault tolerance, and scalability across interconnected systems.

- **Key Features:**
 - Transparency (Access, Location, Replication, Concurrency, and Failure transparency)
 - Scalability
 - Resource sharing
 - Fault tolerance
 - Security

2. Advantages and Types of Distributed Operating Systems

Advantages:

1. **Resource Sharing:** Efficient use of distributed resources.
2. **Scalability:** Systems can grow easily by adding nodes.
3. **Reliability:** Fault tolerance through redundancy.
4. **Speed:** Parallel processing can lead to faster computations.
5. **Flexibility:** Users can access systems from different locations.
6. **Reduced Costs:** Using commodity hardware reduces cost compared to centralized systems.

Types:

1. **Network Operating Systems:**
 - Independent systems communicate and share resources via the network.
 - Each system maintains its own OS.
 - Example: Windows Server, UNIX.
2. **Distributed Operating Systems:**
 - Provides a unified view of the system across multiple nodes.
 - Examples: Amoeba, Plan 9, Chorus.
3. **Multiprocessor Systems:**
 - Shared memory with multiple processors.

- Examples: Symmetric multiprocessing (SMP) systems.

3. Design Issues in Distributed Operating Systems

Communication:

- Ensuring efficient and reliable message passing between nodes.
- Use of protocols like TCP/IP, RPC (Remote Procedure Call), and message queues.

Synchronization:

- Maintaining consistency in a distributed environment.
- Mechanisms include logical clocks (e.g., Lamport Clock), distributed mutual exclusion, and global state.

Process Management:

- Process migration and load balancing.
- Coordination of inter-process communication (IPC).

File System:

- Distributed file systems ensure transparent access to remote files.
- Examples: NFS (Network File System), AFS (Andrew File System).

Fault Tolerance:

- Mechanisms to handle node and communication failures.
- Techniques include replication, checkpointing, and rollback recovery.

Security:

- Ensuring authentication, authorization, and secure communication.

4. Client-Server Computing

Concept:

- A distributed computing model where tasks are divided between service providers (servers) and service requesters (clients).

Key Components:

1. **Client:**
 - Requests services/resources from the server.
 - Examples: Browsers, Mobile Apps.
2. **Server:**

- Provides services/resources to the clients.
- Examples: Web servers, Database servers.

Advantages:

- Centralized control of resources.
- Scalability by adding servers.
- Improved data integrity.

Disadvantages:

- Dependency on server availability.
- Security risks in communication.

Applications:

- Web services, Email systems, Distributed databases, etc.

5. Algorithms for Distributed Control

Mutual Exclusion

In a distributed system, mutual exclusion ensures that only one process can access a critical section (CS) at a time to avoid inconsistencies. Common algorithms include:

1. **Centralized Algorithm:**
 - A central coordinator grants access to the critical section.
 - Simple but has a single point of failure.
2. **Token-Based Algorithm:**
 - A token is passed among processes; possession of the token allows access to the critical section.
 - Efficient but requires token management.
3. **Ricart-Agrawala Algorithm:**
 - A process sends a request message to all other processes before entering the critical section.
 - Access is granted based on timestamp ordering.
 - No central coordinator; suitable for distributed systems.
4. **Maekawa's Algorithm:**
 - Reduces the number of messages by dividing processes into groups.
 - Only a subset of processes need to agree for access.

Deadlock in Distributed Systems

Deadlock occurs when processes are stuck waiting indefinitely for resources held by each other. Common strategies to handle deadlocks:

1. **Deadlock Prevention:**
 - Avoid conditions that lead to deadlock (e.g., ensure no circular wait).
2. **Deadlock Detection:**
 - Periodically check for cycles in the resource allocation graph.

- Algorithms include edge chasing and wait-for graphs.
- 3. **Deadlock Recovery:**
 - Resolve deadlocks by aborting or rolling back one or more processes.
- 4. **Distributed Deadlock Detection:**
 - Uses techniques like probe-based detection, where a deadlock is identified when probes return to the initiating process.

Distributed File System

A Distributed File System (DFS) allows multiple users on a network to access and share files as if they were on their local system.

Characteristics:

- Transparency (Location, Access, Replication, and Performance transparency)
- Scalability
- Fault tolerance
- Security

Examples:

- **NFS (Network File System):** A popular client-server-based file-sharing system.
- **HDFS (Hadoop Distributed File System):** Designed for big data processing.
- **AFS (Andrew File System):** Known for scalability and security.

Key Concepts:

1. **File Replication:**
 - Storing multiple copies of a file across nodes to improve reliability and access speed.
2. **File Locking:**
 - Ensures consistency during concurrent access.
3. **Caching:**
 - Improves performance by temporarily storing copies of frequently accessed files.

6. Security in Distributed Systems

Key Challenges:

1. **Authentication:**
 - Verifying the identity of users and devices.
 - Techniques: Passwords, Certificates, Biometrics.
2. **Authorization:**
 - Ensuring users have permission to access resources.
 - Role-based and attribute-based access control.
3. **Data Integrity:**
 - Preventing unauthorized modifications.
 - Methods: Checksums, Hash functions (e.g., SHA-256).
4. **Confidentiality:**
 - Protecting data from unauthorized access.

- Techniques: Encryption (e.g., AES, RSA).
- 5. **Non-Repudiation:**
 - Ensuring actions cannot be denied by the initiator.
 - Tools: Digital signatures.
- 6. **Fault Tolerance and Recovery:**
 - Protecting against attacks and failures.

Common Security Mechanisms:

- **Encryption:** Secures data in transit and at rest.
- **Firewalls:** Filters unauthorized access.
- **Intrusion Detection Systems (IDS):** Monitors for suspicious activities.
- **Secure Communication Protocols:** HTTPS, SSL/TLS.